

STAR RAG Engine - System Design Document

Abhishek Kulkarni

April 23, 2026

<https://github.com/abhishek-kulkarni>

1. System Overview	3
2. Use Cases	3
3. Requirements	3
3.1 Functional Requirements	3
3.2 Non-Functional Requirements (NFRs)	3
4. Architectural Principles	4
4.1 System Architecture Diagram	4
4.2 Core Infrastructure Stack	5
5. Component Deep Dive	5
5.1 The Web/API Layer (FastAPI)	5
5.2 The Asynchronous Ingestion Pipeline (Celery + Redis)	5
5.3 The Storage Layer	5
5.4 The LLM Orchestrator & Plan Artifacts	6
6. System Data Flows	6
6.1 Asynchronous Document Ingestion Flow (Chained Tasks)	6
6.2 Flow 2: Synchronous RAG Query (Generation)	7
7. Client-API Interaction & Multi-Tenancy	8
7.1 Authentication & Multi-Tenancy Security Boundary	8
7.2 Interaction 1: The Asynchronous Upload Flow (Fire, Forget, Poll)	9
7.3 Interaction 2: The Synchronous Generation Flow (RAG)	9
8. Asynchronous Task Chaining (Celery)	9
8.1 The Claim Check Integration	9
8.2 Step-Level Fault Isolation	10
8.3 State Machine Synchronization	10
9. Parsing & Semantic Chunking Strategy	10
9.1 Phase 1: Text Extraction (DocumentParser)	10
9.2 Phase 2: Semantic Overlap Chunking (SemanticChunker)	10
10. The LLM Orchestrator & RAG Retrieval	11
10.1 Secure Semantic Retrieval (pgvector)	11
10.2 Prompt Architecture & Plan Artifact Injection	11
10.3 Deterministic Structured Output	12
11. Technical Trade-offs	12
Appendix	12
Appendix A: Data Contracts, Schemas and Interfaces	12
Appendix B: Infrastructure, Operations, and Economics	17
Appendix C: System Artifacts & Examples	22
Appendix D: AI Evaluation & Quality Assurance	23
Appendix E: Data Privacy & Retention Strategy	24
Appendix F: Responsible AI (RAI) & System Guardrails	24

1. System Overview

The STAR RAG Engine is an AI-driven backend application designed to ingest unstructured technical documents (PR-FAQs, System Designs, Incident Reports) and highly structured "Plan Artifacts." It uses this data to generate context-aware interview preparation materials, such as STAR-format behavioral answers and resume bullets.

The system utilizes a strict **Multi-Tenant** Retrieval-Augmented Generation (RAG) architecture to ensure AI responses are perfectly isolated per user and are strictly grounded in the user's actual engineering experience while minimizing API token costs and latency.

2. Use Cases

- **Resume Generation:** A user uploads multiple project design docs and requests bullet points highlighting specific skills (e.g., scalability, cloud infrastructure) formatted for an ATS-friendly resume.
- **Behavioral Interview Prep:** A user asks, "Tell me about a time you handled a production outage," and the system retrieves context from past incident reports to generate a STAR-format response.
- **Plan Artifact Enforcement:** A user uploads a strict grading rubric or behavioral framework (Plan Artifact) which acts as immutable instructions for the LLM, guaranteeing responses follow the exact required structure.

3. Requirements

3.1 Functional Requirements

- **Document Ingestion:** The system must accept uploads of PDFs, DOCX, and TXT files.
- **Categorization:** The system must allow users to tag uploads as either STANDARD_DOC (for context retrieval) or PLAN_ARTIFACT (for rule enforcement).
- **Semantic Chunking & Embedding:** STANDARD_DOC files must be parsed, chunked semantically, vectorized, and stored in a vector database.
- **Job Tracking:** The system must provide endpoints to track the asynchronous status of document ingestion workflows (e.g., PENDING, PARSING, CHUNKING, EMBEDDING, COMPLETED).
- **Contextual Generation:** The system must retrieve relevant context using semantic similarity and generate structured output via an LLM.

3.2 Non-Functional Requirements (NFRs)

- **Performance:** Synchronous RAG query responses should begin streaming back to the client within 2 seconds.
- **Scalability & Resilience:** Document ingestion must be handled asynchronously via a

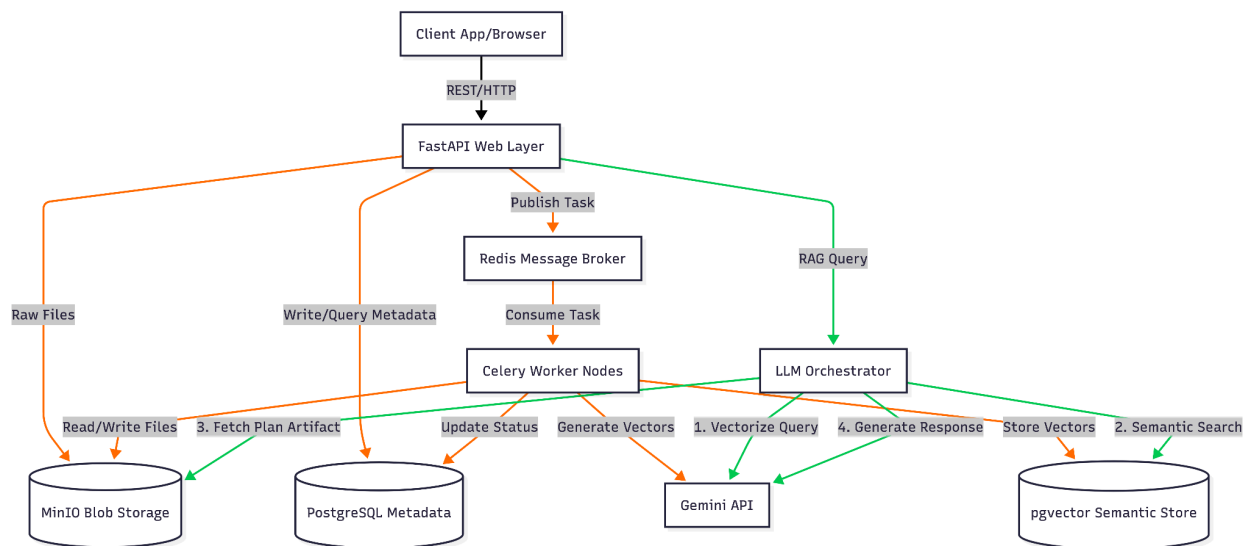
task queue (Celery Chains) to prevent blocking the web server. Failures must be isolated to the specific step to avoid redundant processing.

- **Security (Multi-Tenancy):** Every document, vector, and job must be strictly bound to an authenticated user_id. Cross-tenant data leakage must be cryptographically and logically impossible.
- **Extensibility:** The LLM service layer must be abstracted to allow swapping between different AI providers (e.g., Gemini, Claude, OpenAI) with minimal code changes.
- **Cost Optimization:** The system must utilize RAG over massive context windows to limit token costs, and be fully deployable locally via Docker Compose for zero-cost development.

4. Architectural Principles

- **Local-First, Cloud-Ready:** Infrastructure relies on containerized, open-source equivalents of AWS managed services. This allows for zero-cost local development via Docker Compose while maintaining 1:1 compatibility for future enterprise cloud deployment.
- **Decoupled Workflows:** Heavy data processing (document ingestion) is strictly separated from synchronous client operations (querying and status checks) using message queues and background workers.
- **Polyglot Persistence:** Data storage is optimized by type: Blob storage for raw files, Relational for metadata, and Vector for semantic search.

4.1 System Architecture Diagram



4.2 Core Infrastructure Stack

Component	Technology	AWS Enterprise Equivalent
API Gateway / Web Layer	FastAPI (Python) + Uvicorn	Amazon API Gateway + ECS Fargate
Message Broker / Queue	Redis	Amazon SQS
Async Worker Nodes	Celery	AWS Step Functions + Lambda
Blob Storage	MinIO	Amazon S3
Database (Relational & Vector)	PostgreSQL + pgvector	Amazon Aurora Serverless (pgvector)
LLM / AI Provider	Google Gemini API	Amazon Bedrock

5. Component Deep Dive

5.1 The Web/API Layer (FastAPI)

- Exposes synchronous REST endpoints for the client UI.
- Handles stateless authentication and request payload validation via strict Pydantic schemas.
- Immediately delegates heavy processing tasks to the message queue and returns an HTTP 202 Accepted status to prevent server blocking.

5.2 The Asynchronous Ingestion Pipeline (Celery + Redis)

- Implements a state-machine-like workflow for processing documents.
- Utilizes a Dead Letter Queue (DLQ) mechanism for failed jobs, ensuring dropped documents can be investigated and retried.
- **The Claim Check Pattern:** To avoid passing massive text payloads through the Redis message broker, workers pass lightweight MinIO storage URIs.

5.3 The Storage Layer

- **MinIO (Blob):** Stores raw user uploads (PDFs, DOCX) and "Plan Artifacts."
- **PostgreSQL (Metadata):** Tracks user accounts, document ingestion statuses, and generated artifacts.
- **pgvector (Semantic):** Stores the 1,536-dimensional float arrays representing the text chunks.

5.4 The LLM Orchestrator & Plan Artifacts

- Interfaces directly with the **Google Gemini API**, leveraging *text-embedding-004* for vectorization and *gemini-1.5-flash/pro* for high-fidelity text generation.
- Manages complex prompt engineering logic, enforcing **strict JSON output schemas** to ensure the frontend consumes deterministic and predictable data structures.
- Differentiates "Plan Artifacts" from standard context by injecting them into the LLM's **System Instructions**, establishing them as immutable constraints for reasoning and execution.

5.4.1 Deep Dive: Plan Artifacts

Plan Artifacts are explicit business logic representations, execution rules, or highly structured schemas (e.g., JSON logic, strict rubric requirements) provided by the user to guide the AI's reasoning. They are handled fundamentally differently than standard context documents:

- **Storage & Tagging:** Plan Artifacts are saved to MinIO, but in PostgreSQL, they are tagged explicitly with type: `PLAN_ARTIFACT`. They are *not* chunked or embedded into the vector database.
- **Prompt Injection (Immutable Rules):** During artifact generation, standard documents are fetched via semantic search and placed in the "Context" section of the prompt. Plan Artifacts, however, are injected directly into the LLM's **System Instructions**. This treats them as immutable constraints and execution plans that the LLM must follow, rather than optional background knowledge.
- **Use Case:** If a user uploads a Plan Artifact detailing exactly how an Amazon Principal Engineer's resume bullet must be structured (Situation, Task, Action, Result, plus strict leadership principle mapping), the Orchestrator forces the LLM to execute this exact plan on the retrieved context.

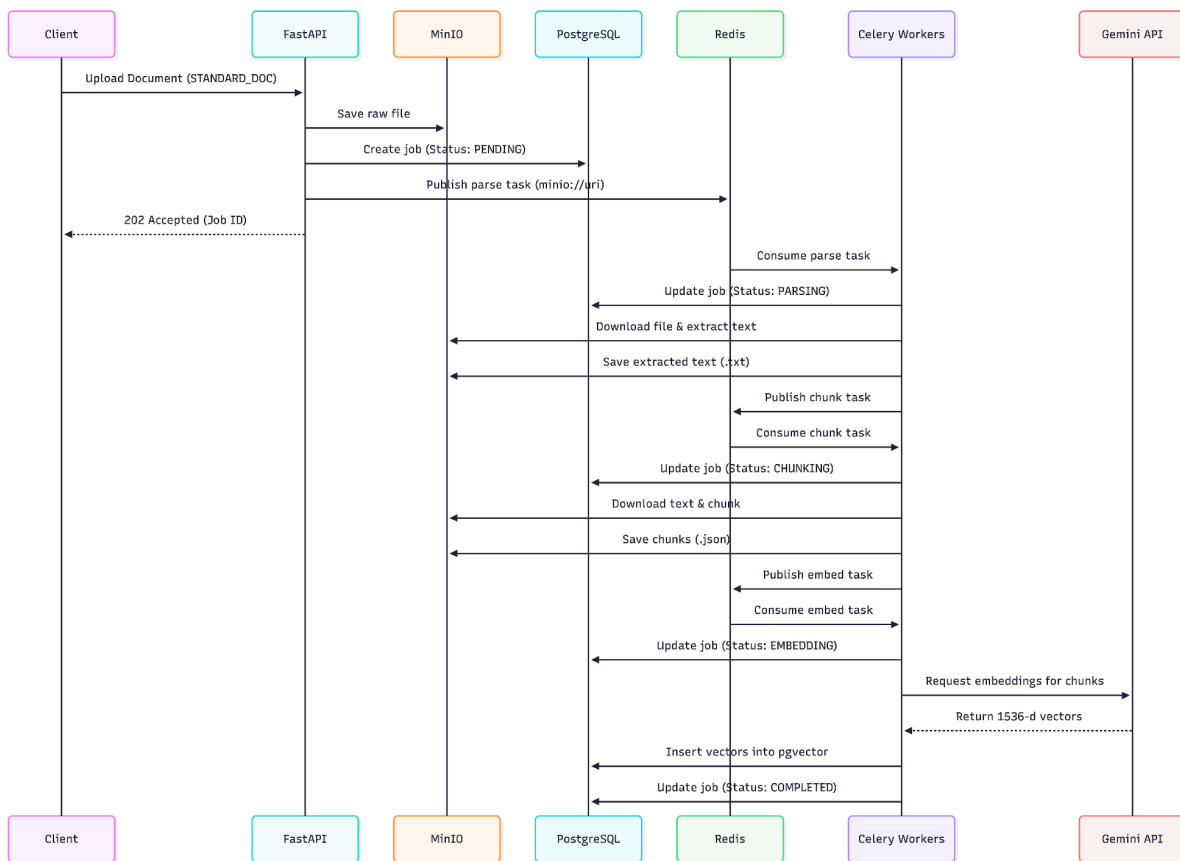
6. System Data Flows

6.1 Asynchronous Document Ingestion Flow (Chained Tasks)

This asynchronous workflow ensures that the API remains responsive while heavy parsing and embedding operations run in the background.

1. **Client** uploads a file to the API endpoint, designating it as either a `STANDARD_DOC` or `PLAN_ARTIFACT`.
2. **FastAPI** calculates the SHA-256 hash
3. **FastAPI** saves the raw file to MinIO, writes a `PENDING` job to Postgres, and publishes a task to Redis.
4. **Routing:** If it's a Plan Artifact, the job is marked `COMPLETED` in Postgres immediately after parsing. If it's a standard document, it moves to the Chunker and Embedder.
5. **Celery Worker (Embedder)** calls the Gemini API to convert text chunks into

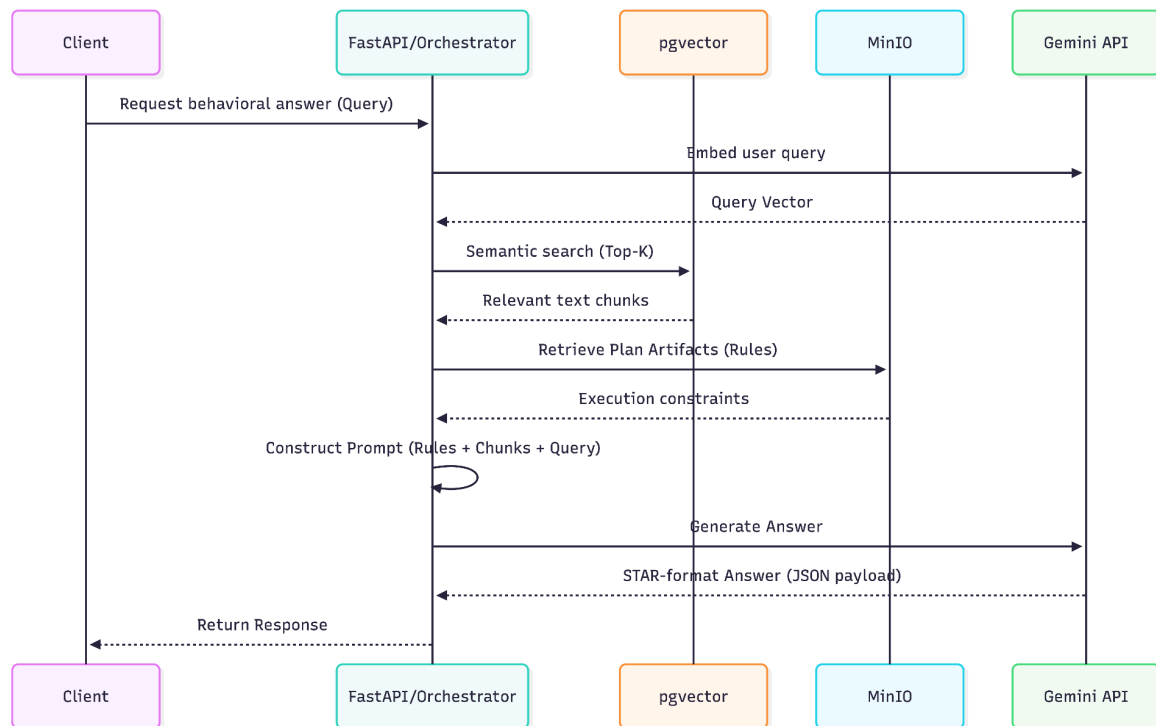
mathematical vectors, writing them to pgvector.



6.2 Flow 2: Synchronous RAG Query (Generation)

This flow outlines how the system orchestrates user queries by combining vector retrieval with immutable plan artifacts.

1. **Client** requests a behavioral answer.
2. **FastAPI** routes the request to the LLM Orchestrator.
3. **Orchestrator** queries pgvector for semantic matches from standard documents.
4. **Orchestrator** retrieves the designated **Plan Artifacts** directly from MinIO via their metadata tags.
5. **Prompt Construction:** The Orchestrator builds the prompt, locking the Plan Artifacts into the System Instructions and placing the retrieved vectors into the Context.
6. **Gemini API** generates the structured response.
7. **FastAPI** streams the payload back to the client.



7. Client-API Interaction & Multi-Tenancy

The interaction between the client application and the STAR RAG Engine is managed via a RESTful API layer built on FastAPI. This layer acts as the system's gateway, completely abstracting the complexity of the Celery worker nodes and PostgreSQL vector databases from the client application. To ensure a highly responsive user experience during heavy processing, the system employs a mix of synchronous and asynchronous interaction patterns.

7.1 Authentication & Multi-Tenancy Security Boundary

To guarantee absolute data isolation between users, the API layer implements a stateless authentication mechanism.

- Every HTTP request must include a standard JSON Web Token (JWT) in the Authorization: Bearer header.
- The API utilizes FastAPI's Dependency Injection system (Depends) to intercept requests before they reach the business logic. It validates the cryptographic signature of the JWT and extracts the user_id (a UUID).
- This user_id is immediately injected into the route controller and bound to all database writes. During retrieval, it acts as a mandatory WHERE clause constraint on the pgvector index, making cross-tenant data leakage impossible.

7.2 Interaction 1: The Asynchronous Upload Flow (Fire, Forget, Poll)

Because parsing and embedding technical documents is computationally expensive, the client never waits for the ingestion process to finish.

- **Upload Request:** The client sends the document and its type (STANDARD_DOC or PLAN_ARTIFACT) via multipart/form-data to POST /documents/upload.
- **Immediate Acknowledgment:** The API saves the raw file, creates a database record, pushes the job to the Celery queue, and immediately returns an HTTP 202 Accepted along with a job_id.
- **Status Polling:** The client UI enters a polling loop (e.g., every 2 seconds), calling GET /documents/{job_id}/status.
- **UI Updates:** As the backend transitions the job state (PENDING -> PARSING -> CHUNKING -> EMBEDDING -> COMPLETED), the frontend updates a visual progress bar for the user.

Ingestion Deduplication

To maintain data integrity and prevent redundant processing, the system implements a robust deduplication strategy during document ingestion. This strategy handles both identical filenames and identical file contents under different names.

Collision Detection Workflow:

1. **Hash Generation:** Upon upload, the system calculates a SHA-256 hash of the file's contents.
2. **Conflict Checking:** The system checks the database for existing documents matching either:
 - a. The exact filename of the uploaded file.
 - b. The generated content_hash (indicating duplicate contents despite potentially different filenames).
3. **Rejection (Default):** If a collision is found, the system rejects the upload and returns an HTTP 409 Conflict status. The frontend alerts the user with a specific error message explaining whether the conflict was due to a matching filename or identical content, and reminds them that they can choose to overwrite the file.

Overwrite & Upsert Behavior:

- The frontend provides an "Overwrite" checkbox for the user.
- When an upload is submitted with the overwrite=true parameter, the system safely replaces the existing document:
 - **Storage Cleanup:** The previous raw file is manually deleted from MinIO storage.
 - **Cascade Deletion:** The system relies on database cascade rules to automatically remove all associated ingestion jobs and embedded chunks from the Vector DB.
 - **Re-ingestion:** The new file is saved to MinIO, a new database record is created

with the updated hash, and a fresh background ingestion task is triggered.

7.3 Interaction 2: The Synchronous Generation Flow (RAG)

For the actual generation of STAR-format answers, the interaction is synchronous.

- **Query Request:** The client sends a JSON payload to POST /generate/star-answer containing the behavioral question.
- **Streaming/Awaiting:** Because the LLM Orchestrator utilizes targeted RAG (retrieving only the top 5 relevant chunks), the context window remains small. The API processes the vector search, builds the prompt with the immutable Plan Artifacts, and returns the strictly formatted JSON response synchronously within a few seconds.

8. Asynchronous Task Chaining (Celery)

The ingestion of technical documents is highly bursty and computationally expensive. To prevent the API Gateway from blocking during large file uploads, the STAR RAG Engine completely decouples ingestion from the web layer using a Celery + Redis asynchronous worker architecture.

To ensure fault tolerance, the pipeline avoids the "Monolithic Task Anti-Pattern." Instead of one massive function that parses, chunks, and embeds, the engine utilizes Celery Task Chains.

8.1 The Claim Check Integration

Because message brokers like Redis are optimized for tiny payloads, passing a 50-page extracted text string between Celery tasks will cause Out-Of-Memory (OOM) crashes. The Task Chain implements the Claim Check pattern: tasks exclusively pass MinIO storage URIs to the next task in the chain.

8.2 Step-Level Fault Isolation

By breaking the pipeline into discrete, chained tasks (parse_document → chunk_document → embed_chunks), the system achieves granular idempotency. If the final embed_chunks task fails due to a Gemini API rate limit, Celery will retry only the embedding task. It utilizes the chunks_uri saved in the Redis state, completely bypassing the expensive need to re-parse or re-chunk the document.

8.3 State Machine Synchronization

Every discrete task in the chain is responsible for updating the IngestionJob state in the PostgreSQL database (e.g., PARSING, CHUNKING). This guarantees that if a worker node unexpectedly dies, the database reflects the exact point of failure, enabling accurate Dead

Letter Queue (DLQ) processing and transparent UI updates for the client.

9. Parsing & Semantic Chunking Strategy

The quality of a Retrieval-Augmented Generation (RAG) system is strictly bounded by the quality of its text chunks. The STAR RAG Engine employs a two-phase extraction and semantic chunking algorithm to preserve the logical boundaries of engineering documents.

9.1 Phase 1: Text Extraction (DocumentParser)

The system utilizes PyMuPDF (for PDFs) and python-docx (for Word documents) because they operate directly on the underlying document streams rather than relying on OCR, making them exceptionally fast and deterministic. The parser strips out non-semantic noise (e.g., page numbers, excessive whitespace, image bytes) while preserving critical structural markers like paragraph breaks (`\n\n`), which act as hints for the downstream chunker.

9.2 Phase 2: Semantic Overlap Chunking (SemanticChunker)

To prevent the "Lost Context" problem where a logical thought is severed across two chunks, the engine implements a Recursive Character Text Splitting algorithm with an explicit overlap window.

- **Target Chunk Size:** 1,000 characters (optimized for the text-embedding-004 model's token limits and ideal recall density).
- **Overlap Window:** 200 characters.
- **Semantic Hierarchy:** The chunker does not slice mid-word. It attempts to split on the largest possible semantic boundary first (double newlines `\n\n` representing paragraphs), falling back to single newlines (`\n`), then periods (`.`), and finally spaces () only if a single paragraph exceeds the chunk limit.
- **The Overlap Rationale:** By forcing chunks to share 200 characters of context, the engine guarantees that if a vector search retrieves "Chunk B," the LLM also gets the tail end of "Chunk A," providing the necessary lead-in context to understand the engineering scenario.

10. The LLM Orchestrator & RAG Retrieval

The LLM Orchestrator is the execution engine for synchronous client queries. It is responsible for bridging the gap between mathematical vector similarity, immutable business logic (Plan Artifacts), and generative AI. It ensures that the final output is deterministic, highly relevant, and

strictly formatted.

10.1 Secure Semantic Retrieval (pgvector)

When a user submits a query (e.g., "Tell me about a time I led a backend migration"), the Orchestrator first converts this text into a 1,536-dimensional vector using the embedding model. It then executes a Nearest-Neighbor search using Cosine Similarity against the pgvector database.

- **The Security Constraint:** The SQL query strictly joins the DocumentChunk table with the Document table to enforce the user_id filter, guaranteeing the context window only contains the requesting user's data.

10.2 Prompt Architecture & Plan Artifact Injection

Standard RAG systems fail when given strict formatting rules because they place the rules in the standard context window, where the LLM can "forget" or deprioritize them (the "Lost in the Middle" phenomenon). The STAR RAG Engine solves this by strictly separating global constraints from user-specific instructions.

- **The Base Prompt:** The Orchestrator maintains a hardcoded global directive that enforces fundamental AI constraints across all queries (e.g., zero hallucination, baseline STAR formatting).
- **The Dynamic Injection (The Fix):** The Orchestrator queries the database for any documents tagged as `PLAN_ARTIFACT` for the specific user. It downloads these raw text rules from MinIO, concatenates them with the Base Prompt, and injects the combined payload directly into the System Instructions of the Gemini model. This elevates the company-specific Plan Artifacts from "helpful context" to "immutable execution rules."

10.3 Deterministic Structured Output

The frontend requires a predictable payload to render the UI (e.g., specific fields for Situation, Task, Action, Result). The Orchestrator leverages the Gemini API's response_schema capability, passing a strict Pydantic model to the AI. This forces the LLM to return a perfectly formatted JSON object, eliminating the need for brittle Regex parsing on the backend.

11. Technical Trade-offs

- **RAG vs. Large Context Windows:** Instead of passing entire documents to an LLM's massive context window for every query (which is slow and costly), we utilize RAG. This guarantees sub-second retrieval times, minimizes token usage, and significantly reduces the probability of LLM hallucinations.
- **pgvector vs. Managed Vector DBs:** Utilizing Postgres for both relational metadata and vector storage eliminates the "split-brain" problem of keeping two separate databases perfectly synchronized. It simplifies local development and

reduces operational costs.

- **System Instructions vs. Standard Context:** By separating Plan Artifacts into System Instructions, we prevent the "Lost in the Middle" problem where LLMs forget execution instructions if they are buried in standard context retrieval.

Appendix

Appendix A: Data Contracts, Schemas and Interfaces

The following definitions represent the strict data boundaries, storage schemas, and API contracts for the STAR RAG Engine.

A.1 State Management (Enums)

These enums guarantee that the database state machine and API boundaries remain strictly typed, preventing arbitrary strings from corrupting the asynchronous ingestion flow.

```
import enum

class DocumentType(str, enum.Enum):
    STANDARD_DOC = "STANDARD_DOC" # Chunked and embedded for RAG context
    PLAN_ARTIFACT = "PLAN_ARTIFACT" # Injected as immutable System Instructions

class JobStatus(str, enum.Enum):
    PENDING = "PENDING"
    PARSING = "PARSING"
    CHUNKING = "CHUNKING"
    EMBEDDING = "EMBEDDING"
    COMPLETED = "COMPLETED"
    FAILED = "FAILED"
```

A.2 Database Schema (SQLAlchemy + pgvector)

The database schema normalizes file storage metadata, job state tracking, and vector embeddings. It explicitly implements Multi-Tenancy via the `user_id` index.

```
from datetime import datetime, UTC
from sqlalchemy import String, Integer, DateTime, ForeignKey, Enum, Text
from sqlalchemy.orm import DeclarativeBase, Mapped, mapped_column, relationship
from pgvector.sqlalchemy import Vector

class Base(DeclarativeBase):
    pass

class Document(Base):
    __tablename__ = "documents"

    id: Mapped[int] = mapped_column(primary_key=True, autoincrement=True)
```

```

# Multi-Tenancy Lock: Indexed for blazing fast RAG retrieval per user
user_id: Mapped[str] = mapped_column(String(128), index=True, nullable=False)

# Used for duplicate content detection and collision avoidance
content_hash: Mapped[str] = mapped_column(String(64), index=True, nullable=False)

filename: Mapped[str] = mapped_column(String(255), nullable=False)
doc_type: Mapped[DocumentType] = mapped_column(Enum(DocumentType),
nullable=False)

# Claim Check URI pointing to the raw file in Blob Storage
minio_raw_uri: Mapped[str] = mapped_column(String(512), nullable=False)
created_at: Mapped[datetime] = mapped_column(default=lambda: datetime.now(UTC))

job: Mapped["IngestionJob"] = relationship("IngestionJob",
back_populates="document", uselist=False)
chunks: Mapped[list["DocumentChunk"]] = relationship("DocumentChunk",
back_populates="document")

class IngestionJob(Base):
    __tablename__ = "ingestion_jobs"

    id: Mapped[int] = mapped_column(primary_key=True, autoincrement=True)
    document_id: Mapped[int] = mapped_column(ForeignKey("documents.id"), unique=True)

    status: Mapped[JobStatus] = mapped_column(Enum(JobStatus),
default=JobStatus.PENDING)
    error_message: Mapped[str | None] = mapped_column(Text, nullable=True)

    # Observability Timestamps for latency tracking per pipeline phase
    created_at: Mapped[datetime] = mapped_column(default=lambda: datetime.now(UTC))
    parsed_at: Mapped[datetime | None] = mapped_column(DateTime(timezone=True),
nullable=True)
    chunked_at: Mapped[datetime | None] = mapped_column(DateTime(timezone=True),
nullable=True)
    completed_at: Mapped[datetime | None] = mapped_column(DateTime(timezone=True),
nullable=True)

    document: Mapped["Document"] = relationship("Document", back_populates="job")

class DocumentChunk(Base):
    __tablename__ = "document_chunks"

    id: Mapped[int] = mapped_column(primary_key=True, autoincrement=True)
    document_id: Mapped[int] = mapped_column(ForeignKey("documents.id"))
    chunk_index: Mapped[int] = mapped_column(Integer, nullable=False)

    text_content: Mapped[str] = mapped_column(Text, nullable=False)

    # Semantic Search payload (Mapped to Gemini's 768-dimension text-embedding-004
model)
    embedding: Mapped[list[float]] = mapped_column(Vector(768))

    document: Mapped["Document"] = relationship("Document", back_populates="chunks")

```

A.3 API Contracts (Pydantic)

These schemas act as the strict validation layer for the FastAPI application, ensuring deterministic data flows in and out of the RAG system.

```
from pydantic import BaseModel, ConfigDict
from datetime import datetime

# --- Client Requests ---

class RAGQueryRequest(BaseModel):
    query: str
    include_plan_artifacts: bool = True

# --- API Responses ---

class IngestionJobResponse(BaseModel):
    id: int
    status: JobStatus
    error_message: str | None
    created_at: datetime
    parsed_at: datetime | None
    chunked_at: datetime | None
    completed_at: datetime | None

    model_config = ConfigDict(from_attributes=True)

class DocumentUploadResponse(BaseModel):
    document_id: int
    filename: str
    doc_type: DocumentType
    job: IngestionJobResponse
    message: str

class STARAnswerResponse(BaseModel):
    """
    Enforced structure for the LLM Orchestrator. The AI is restricted
    to outputting valid JSON matching these exact keys.
    """
    situation: str
    task: str
    action: str
    result: str
    citations: list[int] # Array of chunk IDs used to ground the answer
```

A.4 Asynchronous Worker Interfaces (Celery Chains)

To ensure fault tolerance, the ingestion pipeline is broken down into discrete functions connected via a Celery Chain. If the embedding step fails due to API rate limits, the system

retries *only* the embedding step, skipping the expensive parsing and chunking phases.

```
from celery import shared_task, chain

# --- Worker Task Signatures ---

@shared_task(bind=True, max_retries=3)
def parse_document_task(self, raw_uri: str, job_id: int) -> str:
    """Consumes raw MinIO URI, extracts text, returns parsed_text_uri."""
    pass

@shared_task(bind=True, max_retries=3)
def chunk_document_task(self, parsed_text_uri: str, job_id: int) -> str:
    """Consumes parsed text URI, applies semantic chunking, returns
    chunks_array_uri."""
    pass

@shared_task(bind=True, max_retries=5)
def embed_chunks_task(self, chunks_array_uri: str, job_id: int) -> bool:
    """Consumes chunks URI, generates Gemini embeddings, persists to pgvector."""
    pass

# --- Pipeline Orchestration ---

def trigger_ingestion_pipeline(raw_uri: str, job_id: int):
    """
    Defines the exact sequence of the Celery Task Chain.
    The output of one task is implicitly passed as the first argument to the next.
    """
    workflow = chain(
        parse_document_task.s(raw_uri, job_id),
        chunk_document_task.s(job_id),
        embed_chunks_task.s(job_id)
    )
    workflow.apply_async()
```

A.5 LLM Prompt Configuration (The "Prompt Contract")

This configuration defines the strict boundary between the system's global persona and the user's dynamic evaluation frameworks.

```
# --- Base System Instructions (Global Constraints) ---
BASE_INTERVIEW_COACH_PROMPT = """
You are an expert technical interview coach for Senior and Principal Software
Engineers.
Your core directive is to generate behavioral interview responses based STRICTLY on
the provided context chunks.

GLOBAL CONSTRAINTS:
1. Do not hallucinate external metrics, technologies, or experiences.
2. If the context lacks sufficient detail for a complete answer, state: "Insufficient
context provided."
```


3. Always format your baseline response using the Situation, Task, Action, Result (STAR) framework.

```
"""

def construct_system_instructions(plan_artifacts_text: str | None) -> str:
    """Injects immutable user rules into the core system prompt."""
    if plan_artifacts_text:
        return f"{BASE_INTERVIEW_COACH_PROMPT}\n\nUSER-SPECIFIC EXECUTION RULES:\n{plan_artifacts_text}"
    return BASE_INTERVIEW_COACH_PROMPT

def construct_user_prompt(user_query: str, retrieved_chunks: list[dict]) -> str:
    """Combines semantic context from pgvector with the user's question."""
    context_string = "\n\n".join(
        [f"[Source: {c['filename']} (ID: {c['chunk_id']})]\n{c['text']}" for c in retrieved_chunks]
    )

    return f"""
BACKGROUND CONTEXT:
{context_string}

USER QUESTION:
{user_query}
"""
```

A.6 API Routing Interface

```
from fastapi import APIRouter, Depends, UploadFile, File, Form
from app.models.schemas import DocumentUploadResponse, IngestionJobResponse, STARAnswerResponse, RAGQueryRequest
from app.api.dependencies import get_current_user_id

router = APIRouter()

# --- Ingestion Endpoints ---

@router.post("/documents/upload", response_model=DocumentUploadResponse)
async def upload_document(
    file: UploadFile = File(...),
    doc_type: str = Form(...),
    overwrite: bool = Form(False),
    user_id: str = Depends(get_current_user_id) # Multi-tenancy guardrail
):
    pass

@router.get("/documents/{job_id}/status", response_model=IngestionJobResponse)
async def get_job_status(
    job_id: int,
    user_id: str = Depends(get_current_user_id)
):
    pass
```

```
# --- Generation Endpoint ---

@router.post("/generate/star-answer", response_model=STARAnswerResponse)
async def generate_answer(
    request: RAGQueryRequest,
    user_id: str = Depends(get_current_user_id)
):
    pass
```

Appendix B: Infrastructure, Operations, and Economics

B.1 Local Infrastructure Blueprint (Docker Compose)

Example docker-compose.yml to guarantee 1:1 local-to-cloud infrastructure parity. It also orchestrates the API gateway, the asynchronous worker nodes, and the polyglot persistence layer (relational, vector, and blob storage) with explicit health checks and deterministic startup ordering.

version: '3.8'

```
# --- Common Environment Variables (YAML Anchor) ---
# This keeps the configuration DRY between the API and Worker containers
x-app-env: &app-env
  ENVIRONMENT: local
  # Database Config
  POSTGRES_USER: admin
  POSTGRES_PASSWORD: securepassword
  POSTGRES_DB: starrag
  POSTGRES_HOST: postgres
  POSTGRES_PORT: 5432
  # Message Broker Config
  REDIS_URL: redis://redis:6379/0
  # Storage Config
  MINIO_ENDPOINT: minio:9000
  MINIO_ROOT_USER: minioadmin
  MINIO_ROOT_PASSWORD: minioadmin
  # AI Provider
  GEMINI_API_KEY: ${GEMINI_API_KEY:-your_api_key_here}

services:
  # --- 1. The Persistence Layer ---

  postgres:
    image: pgvector/pgvector:pg16
    container_name: starrag-postgres
    environment:
      POSTGRES_USER: admin
      POSTGRES_PASSWORD: securepassword
      POSTGRES_DB: starrag
```

```

ports:
  - "5432:5432"
volumes:
  - pgdata:/var/lib/postgresql/data
healthcheck:
  test: ["CMD-SHELL", "pg_isready -U admin -d starrag"]
  interval: 5s
  timeout: 5s
  retries: 5

minio:
  image: minio/minio
  container_name: starrag-minio
  command: server /data --console-address ":9001"
  environment:
    MINIO_ROOT_USER: minioadmin
    MINIO_ROOT_PASSWORD: minioadmin
  ports:
    - "9000:9000"    # API Port
    - "9001:9001"    # UI Console Port
  volumes:
    - miniodata:/data
  healthcheck:
    test: ["CMD", "curl", "-f", "http://localhost:9000/minio/health/live"]
    interval: 10s
    timeout: 5s
    retries: 3

redis:
  image: redis:7-alpine
  container_name: starrag-redis
  ports:
    - "6379:6379"
  healthcheck:
    test: ["CMD", "redis-cli", "ping"]
    interval: 5s
    timeout: 3s
    retries: 5

# --- 2. The Compute Layer ---

api:
  build: .
  container_name: starrag-api
  command: uvicorn app.main:app --host 0.0.0.0 --port 8000 --reload
  ports:
    - "8000:8000"
  environment:
    <<: *app-env
  volumes:
    - ./app # Hot-reloading for local development
  depends_on:
    postgres:
      condition: service_healthy

```

```

    redis:
      condition: service_healthy
    minio:
      condition: service_healthy

celery_worker:
  build: .
  container_name: starrag-worker
  command: celery -A app.workers.tasks worker --loglevel=INFO
  environment:
    <<: *app-env
  volumes:
    - ./app
  depends_on:
    redis:
      condition: service_healthy
    postgres:
      condition: service_healthy
    minio:
      condition: service_healthy

volumes:
  pgdata:
    name: starrag-pgdata
  miniodata:
    name: starrag-miniodata

```

B.2 Observability & Tracing (Structured Logging)

In a distributed asynchronous system (FastAPI -> Redis -> Celery), if a document fails to parse, standard print() statements are useless. Adding a structured logging contract shows you know how to trace a request across different machines.

```

// Example of a structured log payload emitted by the Celery Worker
// This allows tools like Datadog or ELK stack to trace the exact lifecycle of a
document.
{
  "timestamp": "2026-04-24T18:13:10Z",
  "level": "ERROR",
  "service": "celery-worker-node-1",
  "trace_id": "req-9f8a-4b2c-11ed",
  "user_id": "usr_773bf82",
  "job_id": 1045,
  "phase": "EMBEDDING",
  "message": "Gemini API rate limit exceeded. Retrying task.",
  "metadata": {
    "retry_count": 2,
    "chunk_array_uri": "minio://chunks/usr_773bf82/chunks_1045.json"
  }
}

```

B.3 Cost Estimation & Token Economics

--- Base Assumptions per User Document ---

Average Technical Doc: 10 pages (~5,000 words / ~6,500 tokens)

Chunk Size: 1,000 characters (~250 tokens per chunk)

Total Chunks per Doc: ~26 chunks

--- Ingestion Costs (Gemini text-embedding-004) ---

Cost per 1M tokens: \$0.00 (Currently free/fractional in many tiers, but assumed negligible)

Vector DB Storage: ~3KB per 1536-d vector -> 26 chunks = ~78KB per document in pgvector.

Storage Impact: Highly efficient. 10,000 documents require < 1GB of vector memory.

--- Generation Costs (Gemini 1.5 Flash - Synchronous RAG) ---

Prompt Tokens:

System Instructions (Plan Artifact): ~500 tokens

Top-5 Retrieved Context Chunks: 5 * 250 = 1,250 tokens

User Query: ~50 tokens

Total Input Context: ~1,800 tokens

Output Tokens:

STAR-Format JSON Response: ~400 tokens

Conclusion: By utilizing RAG instead of stuffing entire 10-page documents into the context window for every query, we reduce generation token costs by ~72% per API call while dramatically reducing latency.

B.4 Vector Index Maintenance (pgvector)

The system utilizes an **HNSW (Hierarchical Navigable Small World)** index on the `DocumentChunk.embedding` column to guarantee sub-millisecond similarity searches.

However, the system's high-churn nature (frequent document ingestion and hard deletions for data privacy) causes the HNSW graph to degrade over time via tombstones and unbalanced node links.

To maintain optimal recall accuracy and query latency, the system implements a Zero-Downtime Maintenance Strategy:

1. **Declarative Partitioning:** The DocumentChunk table is partitioned by user_id (using PostgreSQL LIST or HASH partitioning). This ensures that heavy ingestion by one tenant does not bloat the index of another.
2. **Concurrent Reindexing:** Standard VACUUM operations clear dead tuples, but do not re-optimize the HNSW graph. A background Celery beat task runs a weekly scheduled maintenance job executing REINDEX INDEX CONCURRENTLY on partitions that have exceeded a specific churn threshold (e.g., > 15% row modification).
3. **Zero-Downtime Hot Swapping:** By using the CONCURRENTLY keyword, PostgreSQL builds the new HNSW graph in the background without acquiring exclusive write locks. Once the new graph is built, it is hot-swapped, ensuring the API remains 100% available for synchronous RAG generation.

Operational SQL Example:

```
-- Executed asynchronously by the Celery Beat maintenance worker
REINDEX INDEX CONCURRENTLY idx_user_partition_123_embedding;
```

B.5 Distributed State Reconciliation (The Dual Write Problem)

The asynchronous ingestion pipeline relies on two separate persistence systems: PostgreSQL (for state tracking) and Redis (as the message broker). Because these systems do not share a distributed transaction coordinator, the architecture must account for the Dual Write Problem (e.g., a database commit succeeds, but the application crashes before the message is published to the broker, resulting in a "Ghost Record" stuck in PENDING).

To guarantee eventual consistency, the system cannot rely solely on Dead Letter Queues (DLQs), as DLQs only catch messages that successfully reached the broker. Instead, it employs two state-reconciliation mitigation layers:

1. **Synchronous Fail-Fast (API Layer):** The FastAPI gateway wraps the Celery `apply_async()` call in an explicit try/catch block monitoring for `kombu.exceptions.OperationalError`. If the broker is unreachable, the API immediately transitions the database record to FAILED and returns an HTTP 503 Service Unavailable to the client.
2. **Asynchronous Sweeper (State Machine Reconciliation):** A Celery Beat cron task runs every 5 minutes to execute garbage collection on orphaned database records. It enforces timeouts based on specific state phases:
 - a. **Queue Timeouts:** Identifies jobs where `Status == PENDING` and `created_at < NOW() - 5 minutes`. This catches Dual-Write failures where the message never reached Redis, or severe queue starvation.
 - b. **Processing Timeouts:** Identifies jobs where `Status IN (PARSING, CHUNKING, EMBEDDING)` and `updated_at < NOW() - 60 minutes`. This catches jobs where a worker node suffered a hard crash (e.g., OOM kill) mid-processing without being able to route the message to the DLQ.

Orphaned records caught by the Sweeper are automatically transitioned to FAILED with a descriptive error message, preventing indefinite UI loading states for the end user and allowing the frontend to prompt a retry.

B.6 Dead Letter Queues & Scalable Alerting

At high ingestion volumes, relying on database polling (`SELECT COUNT(*) WHERE status = 'FAILED'`) for system monitoring is an anti-pattern that creates unnecessary load on the primary PostgreSQL instance. To ensure the STAR RAG Engine remains performant and observable at scale, the architecture decouples transactional state from operational telemetry.

1. **Dead Letter Queues (DLQ) for Poison Messages:** When a Celery task exhausts its maximum retries (e.g., `embed_chunks_task` failing 5 times due to a prolonged Gemini API outage), the system updates the PostgreSQL state to `JobStatus.FAILED` for the user UI. Crucially, it then routes the task payload (the MinIO Claim Check URI) to a dedicated

Redis Dead Letter Queue (starrag_dlq). This prevents unprocessable "poison messages" from clogging the main worker pools while preserving the payload for manual inspection and automated replay once the upstream issue is resolved.

2. **Push-Based Telemetry & Alerting:** Rather than polling the database for failures, the Celery workers are instrumented with Prometheus/StatsD middleware.
 - Every successful task emits a task_success increment.
 - Every failed task emits a task_failed increment, tagged with the phase (e.g., parsing, embedding).
 - An external observability platform (e.g., Datadog, Grafana) scrapes these metrics.

Alarming Strategy: Alerts are strictly threshold-based (e.g., PagerDuty is triggered if the task_failed rate exceeds 5% over a 5-minute rolling window, or if the DLQ depth exceeds 1,000 messages). This ensures high-signal, low-noise monitoring without impacting the transactional database's ability to serve synchronous RAG queries.

Appendix C: System Artifacts & Examples

C.1 Example Plan Artifact: Principal Engineer Evaluation Rubric

This document illustrates a highly opinionated, user-provided execution rubric. When the LLM Orchestrator retrieves this artifact, it bypasses the standard context window and forces the generative AI to adopt these exact Amazon evaluation standards, preventing generic or weakly structured answers.

DOCUMENT TYPE: PLAN_ARTIFACT

TARGET COMPANY: Amazon (AWS)

ROLE LEVEL: L6/L7 (Senior/Principal Software Engineer)

EXECUTION DIRECTIVE:

When generating the STAR response, you must map the provided engineering context to the relevant Amazon Leadership Principles (LPs). The "Action" and "Result" sections must explicitly demonstrate these principles in practice.

LP MAPPING REQUIREMENTS:

1. If the query involves SYSTEM DESIGN OR ARCHITECTURE:
 - Prioritize: "Invent and Simplify" & "Are Right, A Lot".
 - Action Constraint: You must highlight how the architecture reduced complexity or decoupled tight dependencies.
 - You must identify the specific mechanism used to validate the technical design before implementation (e.g., prototyping, load testing).
2. If the query involves PRODUCTION OUTAGES OR INCIDENTS:
 - Prioritize: "Ownership" & "Deliver Results".
 - Action Constraint: Highlight the immediate mitigation steps taken.
 - Result Constraint: You must explicitly state the long-term systemic fix (Correction of Error / COE) implemented to prevent recurrence.
3. If the query involves CROSS-FUNCTIONAL CONFLICT OR ROADMAPS:
 - Prioritize: "Have Backbone; Disagree and Commit" & "Customer Obsession".
 - Action Constraint: Detail the data or customer metrics used to push back against

pushy stakeholders or arbitrary deadlines.

- Action Constraint: Show how consensus was reached and the commitment to the final path.

FORMATTING OVERRIDE:

At the very end of the STAR response, add a new JSON key called "leadership_principles_demonstrated" containing an array of the top 2 LPs shown in the story, along with a 1-sentence justification for each.

Appendix D: AI Evaluation & Quality Assurance

Traditional deterministic unit testing is insufficient for generative AI. To prevent regressions when altering chunk sizes, embedding models, or system prompts, the STAR RAG Engine employs an "LLM-as-a-Judge" evaluation framework (inspired by RAGAS) running on a golden dataset of test queries.

When testing a new configuration, the pipeline measures three specific metrics:

1. Context Precision (Retrieval Quality): Did pgvector retrieve chunks that actually contain the answer, or is the context polluted with irrelevant text?
2. Faithfulness (Anti-Hallucination): Is the generated STAR response strictly derived from the retrieved chunks, or did the LLM invent outside information?
3. Answer Relevance (Generation Quality): Does the response directly answer the user's prompt while adhering to the injected Plan Artifact rules?

By automating this pipeline, we can confidently deploy changes to the chunking algorithm or prompt architecture knowing exactly how it impacts response quality.

D.1 The Golden Dataset Structure

To establish a baseline for the "LLM-as-a-Judge" metrics, the system utilizes a curated **Golden Dataset**. This dataset acts as the deterministic test suite for the non-deterministic generative pipeline.

It consists of hand-crafted test cases covering various behavioral query types (e.g., conflict resolution, system design, outages). Each test case in the dataset contains:

- **User Query:** The mock behavioral interview question.
- **Target Plan Artifact:** The specific company rubric to be injected (e.g., Amazon LPs).
- **Ground Truth Context:** The exact chunk_ids from the test document that the vector database *should* retrieve if the semantic search is functioning optimally.
- **Reference Answer:** A human-verified, perfect STAR response used as the baseline to measure the AI's generated output for accuracy, tone, and formatting.

Whenever the RAG architecture is modified (e.g., tweaking the pgvector HNSW parameters, adjusting semantic chunk overlap, or upgrading the Gemini model version), the pipeline executes this Golden Dataset to mathematically verify that retrieval recall and generation quality have not regressed.

Appendix E: Data Privacy & Retention Strategy

Given the highly sensitive nature of engineering design docs and incident reports, the system enforces a strict data lifecycle.

1. **At-Rest Encryption:** All raw blobs in MinIO and relational/vector data in PostgreSQL are encrypted at rest using AES-256.
2. **Hard Deletion (The Right to be Forgotten):** When a user deletes a document or their account, it triggers a synchronous cascade:
 - a. The raw file is immediately purged from the MinIO bucket.
 - b. The PostgreSQL Document and associated IngestionJob records are deleted.
 - c. Crucially: The 1,536-dimensional vectors are hard-deleted from pgvector and the index is re-vacuumed to ensure no residual semantic data can be retrieved.
 - d. The same synchronous cascade deletion mechanism is also triggered during a file "Overwrite" to guarantee old vectors and files are purged.
3. **Zero-Retention Inference:** API calls to the Gemini provider explicitly opt out of data sharing, ensuring user context is not retained by Google for model training.

Appendix F: Responsible AI (RAI) & System Guardrails

While the AI Evaluation strategy (Appendix D) ensures factual accuracy, the system must also adhere to Responsible AI (RAI) principles to ensure fairness, safety, and transparency, particularly given the sensitive nature of career advancement and proprietary corporate documents.

F.1 Algorithmic Fairness & Bias Mitigation

Generative models can exhibit latent demographic biases, potentially altering the tone or perceived competence of a generated STAR response based on names or pronouns detected in the source documents. To mitigate this:

- **Prompt Guardrails:** The LLM Orchestrator's Base System Prompt includes an explicit anti-bias directive: *"Maintain a strictly objective, highly technical tone. Do not alter the evaluation stringency or behavioral descriptions based on inferred demographic markers, gender, or cultural background."*
- **Evaluation:** The evaluation pipeline periodically runs "Counterfactual Fairness" tests, swapping names/pronouns in the golden dataset to mathematically verify that the generated output remains structurally and tonally identical.

F.2 Pre-Inference PII Sanitization

Uploaded engineering documents (e.g., Incident Reports, system logs) frequently contain sensitive Personally Identifiable Information (PII) such as coworker names, customer emails, or internal IP addresses.

- **Middleware Integration:** Before the text reaches the SemanticChunker or the embedding API, it passes through a sanitization middleware layer (utilizing an open-source NLP

analyzer like Microsoft Presidio).

- **Masking:** Sensitive entities are deterministically replaced with tokens (e.g., <EMAIL_REDACTED>, <PERSON_1>), ensuring that unauthorized PII is never vectorized, stored in pgvector, or sent to the LLM provider.

F.3 Explainability & Provenance

To combat the "Black Box" nature of LLM generation, the system mandates strict provenance. The LLM Orchestrator is restricted by its Pydantic JSON schema to return an array of citations mapped to specific DocumentChunk IDs. The frontend UI leverages this to provide a split-screen view, allowing the user to click a generated sentence and instantly see the highlighted source text it was derived from, ensuring complete transparency in how the AI arrived at its output.